LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 8
Student Name: Gorapalli Sravya
Roll No.: A24126552257
Date: 30-10-2026

---

1. TITLE

Implementation of HTTP Methods Using JSON Server

2. AIM

To implement and test the HTTP methods GET, POST, PUT, PATCH and DELETE on a REST API created with JSON Server, and to observe the status code returned for each request.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the purpose of the HTTP methods GET, POST, PUT, PATCH and DELETE in a REST API.
- To create a fake REST API from a JSON file using JSON Server without writing any server code.
- To send HTTP requests to the API from a Node.js client and read the responses.
- To filter records using a query parameter.
- To interpret HTTP status codes such as 200 OK, 201 Created and 404 Not Found.

4. THEORY

JSON Server is a Node.js tool that turns a simple JSON file into a complete REST API in seconds. Every top-level key in the file (for example "students") becomes a resource, and routes such as /students and /students/:id are generated automatically. Changes made through the API are written back to the JSON file, which makes it useful for prototyping front-end applications before a real back end is ready.

HTTP methods describe the action to perform on a resource. GET retrieves data, POST creates a new record, PUT replaces an existing record completely, PATCH updates only the fields that are sent, and DELETE removes a record. JSON Server also supports filtering through query strings, for example /students?branch=CSE-AIML.

Every response carries a status code: 200 OK means the request succeeded, 201 Created means a new record was created (returned for POST), and 404 Not Found means the requested record does not exist (for example after it has been deleted).

5. ALGORITHM / PROCEDURE

1. Install Node.js and create a project folder.
2. Initialize the project using npm init -y.

3. Install JSON Server using npm install json-server@0.17.4.

4. Create db.json containing a "students" array with four student records.

5. Start the server using npx json-server --watch db.json --port 3000.

6. Create client.js with a helper function that sends an HTTP request and prints the status code and response.

7. Send a GET request to /students to read all records.

8. Send a GET request to /students/2 to read a single record.

9. Send a GET request to /students?branch=CSE-AIML to read filtered records.

10. Send a POST request with a JSON body to create a new student.

11. Send a PUT request to /students/2 to replace that student's record.

12. Send a PATCH request to /students/3 to update only the marks field.

13. Send a DELETE request to /students/1 and confirm with a GET that it now returns 404.

14. Check the request log printed by JSON Server and the final list of students.

6. PROGRAM

Terminal commands

```
npm init -y
npm install json-server@0.17.4
npx json-server --watch db.json --port 3000
node client.js
```

db.json

```
{
  "students": [
    { "id": 1, "rollNo": "23CM001", "name": "Ravi Kumar",    "branch": "CSE-
AIML", "marks": 85 },
    { "id": 2, "rollNo": "23CM002", "name": "Sita Devi",     "branch": "CSE",
"marks": 68 },
    { "id": 3, "rollNo": "23CM003", "name": "Arjun Reddy",   "branch": "ECE",
"marks": 91 },
    { "id": 4, "rollNo": "23CM004", "name": "Lakshmi Priya", "branch": "CSE-
AIML", "marks": 55 }
  ]
}
```

client.js

```
const http = require("http");
const BASE = "http://localhost:3000";

function request(method, path, body) {
  return new Promise((resolve, reject) => {
    const payload = body ? JSON.stringify(body) : null;
    const req = http.request(BASE + path, {
      method,
      agent: false,
      headers: payload ? { "Content-Type": "application/json", "Content-Length":
Buffer.byteLength(payload) } : {},
    }, (res) => {
      let raw = "";
      res.on("data", (c) => (raw += c));
      res.on("end", () => resolve({ status: res.statusCode, statusText:
```

```
res.statusMessage, data: raw ? JSON.parse(raw) : null }));
    });
    req.on("error", reject);
    if (payload) req.write(payload);
    req.end();
  });
}

async function call(method, path, body) {
  console.log(`\n> ${method} ${BASE}${path}` + (body ? `\n  Request body: $
{JSON.stringify(body)}` : ""));
  const { status, statusText, data } = await request(method, path, body);
  console.log(`  Status: ${status} ${statusText}`);
  if (Array.isArray(data)) {
    console.log(`  Response (${data.length} records):`);
    data.forEach((d) => console.log("  " + JSON.stringify(d)));
  } else {
    console.log("  Response: " + JSON.stringify(data));
  }
}

(async () => {
  await call("GET", "/students");                                    // read all
  await call("GET", "/students/2");                                  // read one
  await call("GET", "/students?branch=CSE-AIML");                    // read with
filter
  await call("POST", "/students", { rollNo: "23CM005", name: "Kiran Babu", branch:
"MECH", marks: 77 });      // create
  await call("PUT", "/students/2", { rollNo: "23CM002", name: "Sita Devi", branch:
"CSE-AIML", marks: 74 });  // replace
  await call("PATCH", "/students/3", { marks: 94 });                 // partial
update
  await call("DELETE", "/students/1");                               // delete
  await call("GET", "/students/1");                                  // confirm 404
after delete
  await call("GET", "/students");                                    // final state
})();
```

7. CODE EXPLANATION

Tag / Code	Explanation
json-server --watch db.json	Starts the REST API from db.json and reloads it when the file changes
"students": [...]	Top-level key that becomes the /students resource
GET /students, /students/2	Reads all records or a single record by id
GET /students?branch=...	Filters records using a query parameter
POST /students	Creates a record; JSON Server assigns the next id and returns 201 Created
PUT /students/2	Replaces the whole record with the body that is sent
PATCH /students/3	Changes only the fields included in the body
DELETE /students/1	Removes the record; a later GET on that id returns 404
http.request()	Node.js module used by the client to send each request
JSON.stringify()	Converts the request body object into a JSON string

8. TEST CASES AND EXECUTION DATA

JSON Server was started on port 3000 and client.js sent each request. The status codes and responses shown below are the real output of that run.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	GET /students	200 OK with all students	200 OK, 4 records returned	Pass
TC-02	GET /students/2	200 OK with Sita Devi's record	200 OK, record with id 2 returned	Pass
TC-03	GET /students?branch=CSE-AIML	200 OK with only CSE-AIML students	200 OK, 2 records returned	Pass
TC-04	POST /students with a new student	201 Created with a new id	201 Created, new record added with id 5	Pass
TC-05	PUT /students/2 with a full record	200 OK, record replaced	200 OK, branch and marks replaced	Pass
TC-06	PATCH /students/3 with marks only	200 OK, only marks changed	200 OK, marks changed to 94	Pass
TC-07	DELETE /students/1, then GET /students/1	200 OK for delete, then 404 Not Found	200 OK, then 404 Not Found	Pass

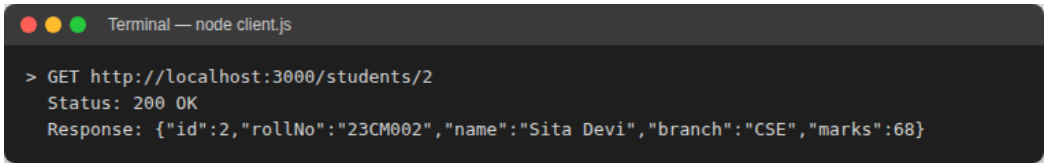
Test Case Output

TC-01 — *200 OK with all students*



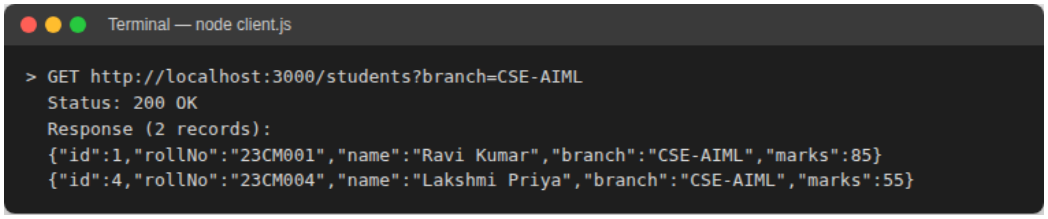
GET /students returns all 4 records

TC-02 — *200 OK with Sita Devi's record*



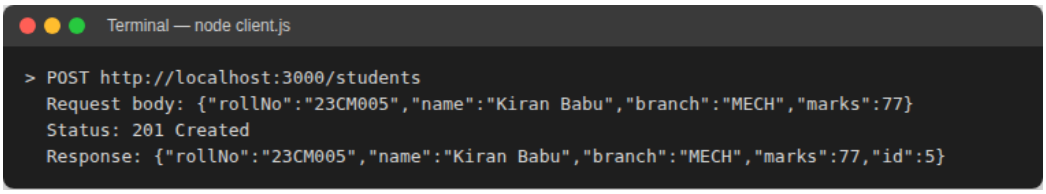
GET /students/2 returns the single matching record

TC-03 — *200 OK with only CSE-AIML students*



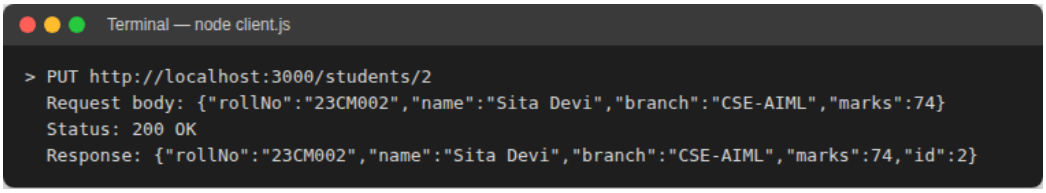
GET /students?branch=CSE-AIML returns the 2 matching records

TC-04 — *201 Created with a new id*



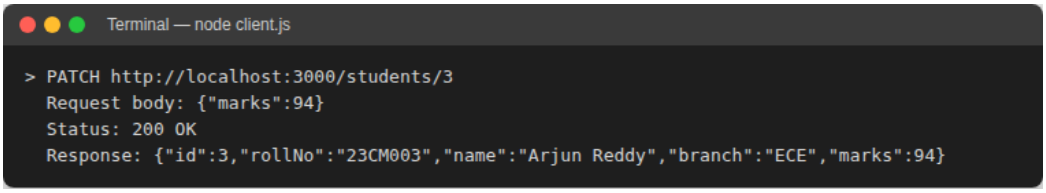
POST /students creates Kiran Babu; the server assigns id 5

TC-05 — *200 OK, record replaced*



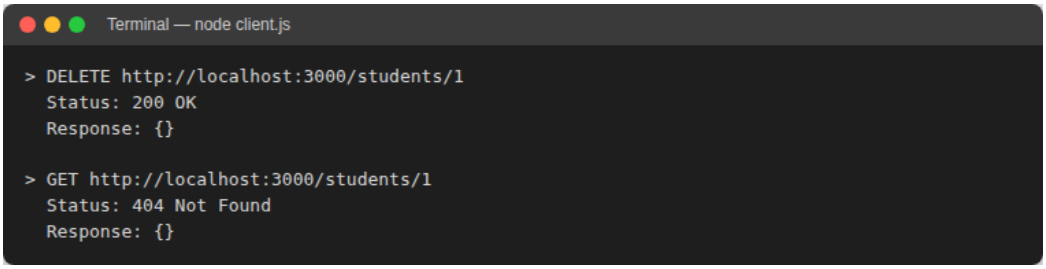
PUT /students/2 replaces the full record

TC-06 — *200 OK, only marks changed*



PATCH /students/3 updates only the marks field

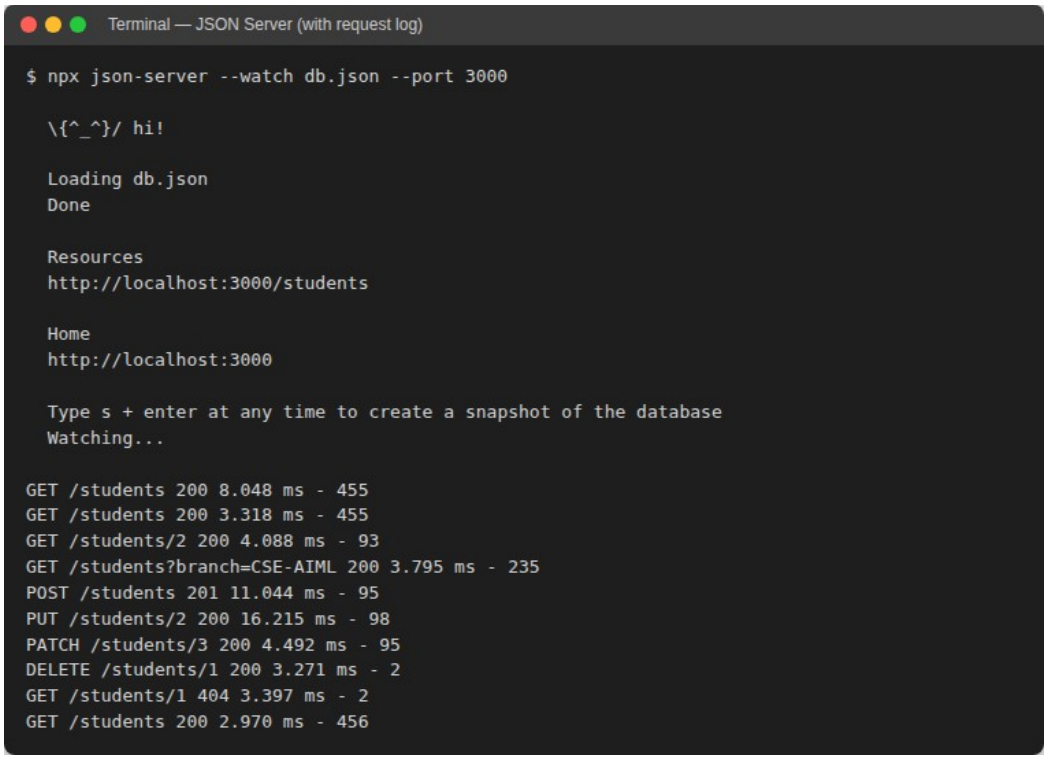
TC-07 — *200 OK for delete, then 404 Not Found*



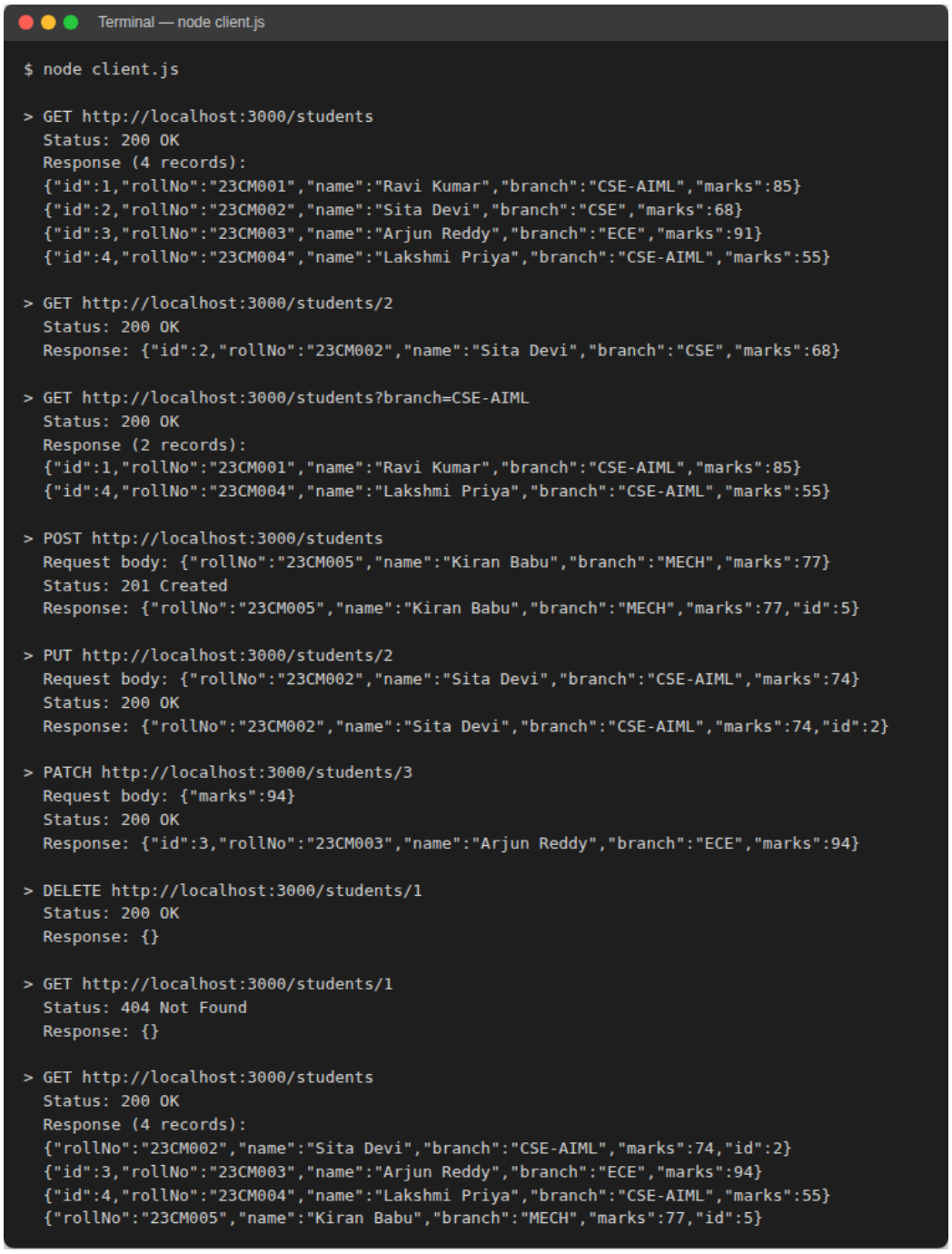
DELETE /students/1 succeeds; fetching the same id afterwards returns 404

9. OUTPUT

Output: JSON Server Request Log and Complete Client Session



JSON Server terminal showing every request received (the first GET /students is the readiness check made by the test script before client.js ran)



Complete output of node client.js covering all HTTP methods

10. RESULT

Thus, the HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented and tested on a REST API created with JSON Server, and the status codes 200, 201 and 404 were observed for the corresponding requests.